

Guia de Deploy em Produção

Sistema de Gestão de Academias
VPS Hostinger + Docker Compose + Cloudflare

Aplicação	Flask (API) + Next.js (Frontend) + Redis + Nginx
Infraestrutura	VPS Hostinger (Ubuntu 22.04) com Docker Compose
Borda	Cloudflare CDN/DNS/TLS (modo Full SSL)
Banco	Supabase gerenciado (fora do Compose)
Data	29/06/2026
Versão	1.0 — gerado automaticamente por scripts/gerar_doc_deploy.py

Documento técnico — uso interno do projeto.

1. Arquitetura de Produção

O sistema é composto por 4 serviços Docker na VPS, com o Supabase gerenciado externamente (não entra no Compose). O Cloudflare fica na borda como CDN, DNS e terminador TLS — a VPS recebe apenas tráfego HTTP de IPs do Cloudflare.

Serviço	Imagem / Build	Porta interna	Função
nginx	nginx:1.27-alpine (imagem oficial)	80 / 443 (host)	Proxy reverso. Roteia <code>/api/*</code> → Flask, <code>/*</code> → Next.js. Servidores estáticos em <code>static</code> .
frontend	Build multi-stage Node 20-alpine	3000 (interno)	Next.js standalone. Serve SSR e assets estáticos. <code>NEXT_PUBLIC_*</code> embutido no build.
backend	Build multi-stage Python 3.11-alpine	5000 (interno)	Flask via gunicorn. API REST. APScheduler (notificações). Conecta ao Supabase.
redis	redis:7-alpine (imagem oficial)	6379 (interno)	Rate limiting compartilhado (Flask-Limiter) e lock do APScheduler (evita e-mails duplicados).

Fluxo de requisição:

1. Usuário acessa `https://academia.seudominio.com.br`
2. Cloudflare termina TLS e repassa HTTP para a VPS na porta 80
3. Nginx decide: `/api/*` vai para Flask (porta 5000); todo o resto vai para Next.js (porta 3000)
4. Next.js renderiza a página (SSR ou cached) e retorna ao browser
5. JavaScript do browser faz fetch para `/api/auth/login` etc. — mesma origem, sem CORS
6. Flask autentica via Supabase Auth e acessa o banco via Supabase SDK

O prefixo `/api/` é removido pelo nginx antes de repassar ao Flask. Exemplo: `GET /api/auth/login` → `GET /auth/login` (Flask nunca vê o prefixo). Por isso o `NEXT_PUBLIC_API_URL` em produção é `https://seudominio.com/api` (com `/api`), mas as rotas do Flask não precisam de prefixo.

2. Pré-requisitos na VPS Hostinger

Todos os comandos abaixo são executados via SSH na VPS como root ou usuário sudo.

2.1 Sistema operacional

Ubuntu 22.04 LTS (recomendado na Hostinger). Verifique:

```
lsb_release -a
```

2.2 Docker e Docker Compose v2

Instale Docker Engine + plugin Compose (v2) com o script oficial:

```
curl -fsSL https://get.docker.com | sh
```

```
docker --version && docker compose version
```

O script acima instala o Docker Engine e o plugin compose. Verifique que o output é Docker Compose version v2.x.x (não v1). Se aparecer docker-compose (com hífen) ainda é a versão v1 — remova e reinstale.

2.3 Git

```
apt-get update && apt-get install -y git
```

2.4 Firewall (UFW)

Libere apenas as portas necessárias. Cloudflare se conecta nas portas 80 e 443.

```
ufw allow 22/tcp ufw allow 80/tcp ufw allow 443/tcp ufw enable ufw status
```

✓ Feche a porta 5000 (Flask), 3000 (Next.js) e 6379 (Redis) — esses serviços comunicam apenas dentro da rede Docker interna (não precisam ser expostos).

2.5 Configuração do Supabase

- Bucket de Storage criado com o nome definido em AVATAR_BUCKET (padrão: avatars) — marcar como **público**.
- Migrations executadas em ordem no SQL Editor do Supabase (pasta backend/migrations/, arquivos 001 a 011).
- E-mail de notificação configurado no painel Supabase Auth (opcional, mas recomendado).
- Chaves copiadas: SUPABASE_URL, SUPABASE_ANON_KEY e SUPABASE_SERVICE_ROLE_KEY do painel Project Settings → API.

3. Deploy Passo a Passo

1 Clonar o repositório na VPS

Escolha um diretório de trabalho (ex.: /opt/academia) e clone:

```
mkdir -p /opt/academia && cd /opt/academia git clone
https://github.com/SEU_USUARIO/gestao-academia.git . # ou com SSH: git clone
git@github.com:SEU_USUARIO/gestao-academia.git .
```

Substitua o URL pelo repositório real. Se o projeto for privado, configure uma deploy key no GitHub: `ssh-keygen -t ed25519 -C 'deploy@vps'` e adicione a chave pública em Settings → Deploy Keys.

2 Criar e preencher o arquivo .env

O `docker-compose.yml` lê todas as variáveis de um único `.env` na raiz do projeto:

```
cd /opt/academia cp .env.production.example .env nano .env
```

Preencha os campos marcados com PREENCHER. Os mínimos obrigatórios são:

<code>FRONTEND_URL</code>	<code></code> URL pública completa, ex.: <code>https://academia.seudominio.com.br</code>
<code>SUPABASE_URL</code>	<code></code> URL do projeto Supabase
<code>SUPABASE_ANON_KEY</code>	<code></code> Chave de acesso anônima
<code>SUPABASE_SERVICE_ROLE_KEY</code>	<code></code> Chave de acesso de serviço (nunca versionar)
<code>EMAIL_USER</code>	<code></code> Conta Gmail para o Mailtrap (Mailtrap)

A variável `SUPABASE_SERVICE_ROLE_KEY` permite acesso total ao banco (ignora RLS). Nunca compartilhe, versione ou imprima nos logs. No `.gitignore`, verifique que `.env` (sem sufixo) está na lista.

3 Build e start dos containers

Na raiz do projeto (onde está o `docker-compose.yml`):

```
cd /opt/academia docker compose up -d --build
```

O primeiro build leva 5–10 minutos (baixa imagens base, compila Python e Next.js). Builds subsequentes são muito mais rápidos (cache de layers).

Se o build do frontend falhar com 'out of memory', adicione swap na VPS: `fallocate -l 2G /swapfile && chmod 600 /swapfile && mkswap /swapfile && swapon /swapfile`

4 Verificar os serviços

```
docker compose ps docker compose logs --tail=50 backend docker compose logs
--tail=50 frontend
```

Todos os serviços devem aparecer como `healthy` ou `running`. O healthcheck do backend checa `GET /health`; o do frontend checa se o servidor retorna HTML na raiz.

```
# Teste rápido: deve retornar {"status": "ok"} curl http://localhost/api/health
```

5 Configurar DNS no Cloudflare

1. Faça login em **dash.cloudflare.com** e selecione seu domínio.
2. Vá em **DNS** → **Records** e adicione um registro **A**: Nome = academia (ou @), Valor = IP público da VPS Hostinger, Proxy = **Laranja (Proxied)**.
3. Vá em **SSL/TLS** → **Overview** e selecione modo **Full** (ou Full Strict se tiver certificado Origin).
4. Em **SSL/TLS** → **Edge Certificates**, ative **Always Use HTTPS** e **Automatic HTTPS Rewrites**.
5. (Opcional) Em **Security** → **WAF**, ative as regras gerenciadas para proteção extra.
6. Aguarde a propagação DNS (geralmente < 5 minutos com Cloudflare).

Modo Cloudflare **Full** (não Flexible): a conexão Cloudflare ↔ VPS é HTTPS, mas com certificado autoassinado. Para **Full Strict**, gere um certificado Cloudflare Origin em SSL/TLS → Origin Server e monte em `nginx/ssl/`.

Com Cloudflare Proxied ativado, a VPS fica protegida atrás dos IPs do Cloudflare. Adicione as regras de firewall da Hostinger para aceitar apenas os IPs do Cloudflare nas portas 80/443 para máxima proteção. ✓

6 Criar o primeiro usuário admin

Com o backend rodando, crie o primeiro usuário admin diretamente no Supabase:

- Painei Supabase → Authentication → Users → **Add user**: e-mail e senha do admin.
- SQL Editor do Supabase — execute o SQL abaixo para criar o perfil admin:

```
INSERT INTO public.profiles (id, nome, tipo, ativo) SELECT id, email, 'admin', true FROM auth.users WHERE email = 'admin@academia.com.br';
```

Substitua o e-mail pelo que foi cadastrado no passo anterior. Os demais usuários (recepcionista, instrutor) podem ser criados via `POST /configuracoes/usuarios` pela interface web.

4. Referência Completa de Variáveis de Ambiente

Todas as variáveis ficam em um único `.env` na raiz do projeto. O `docker-compose.yml` injeta cada variável no serviço correto.

4.1 Geral

Variável	Padrão / Exemplo	Descrição
FRONTEND_URL	<code>https://academia.seu dominio.com</code>	URL pública do frontend. Usada no CORS (Flask) e para derivar <code>NEXT_PUBLIC_API_URL</code> .

4.2 Supabase

Variável	Padrão / Exemplo	Descrição
SUPABASE_URL	<code>https://xxx.supabase.co</code>	URL do projeto. Painel → Project Settings → API.
SUPABASE_ANON_KEY	<code>eyJ...</code>	Chave pública. Segura para ir ao frontend (embutida no build).
SUPABASE_SERVICE_ROLE_KEY	<code>eyJ...</code>	■ SECRETA. Bypassa RLS. Apenas no backend. Nunca versionar.
SUPABASE_JWT_SECRET	(vazio)	Opcional. JWT secret do Supabase. Não usado atualmente para validação local.

4.3 Gunicorn / Performance

Variável	Padrão / Exemplo	Descrição
WEB_CONCURRENCY	2	Nº de workers gunicorn. Regra: $2 \times \text{nº CPUs} + 1$. Com 1 CPU: use 2.

4.4 Rate Limiting

Variável	Padrão / Exemplo	Descrição
RATELIMIT_DEFAULT	200 per minute	Limite global por IP/token. Redis compartilha entre workers.
RATELIMIT_LOGIN	10 per minute;50 per hour	Limite específico do <code>/auth/login</code> (anti brute-force).

4.5 Avatares

Variável	Padrão / Exemplo	Descrição
AVATAR_BUCKET	avatars	Nome do bucket no Supabase Storage (deve ser público).
AVATAR_MAX_BYTES	4194304	Tamanho máximo do upload em bytes (4 MB).
AVATAR_SIZE_PX	512	Tamanho máximo em pixels após redimensionamento (quadrado).

4.6 E-mail (Gmail)

Variável	Padrão / Exemplo	Descrição
EMAIL_HOST	smtp.gmail.com	Servidor SMTP. Para outro provedor, ajuste EMAIL_PORT também.
EMAIL_PORT	587	Porta SMTP com STARTTLS. Gmail usa 587. SSL direto: 465.
EMAIL_USER	academia@gmail.com	Conta Gmail que envia os e-mails.
EMAIL_PASSWORD	xxxx xxxx xxxx xxxx	■ Senha de app (16 chars). Não é a senha normal da conta.
EMAIL_FROM	academia@gmail.com	Endereço de remetente exibido no e-mail. Geralmente = EMAIL_USER.

5. O que Muda do Desenvolvimento para Produção

5.1 Backend (.env)

Variável	Dev → Produção	Impacto
RATELIMIT_STORAGE_URI	memory:// → Redis automático	Configurado automaticamente pelo docker-compose como redis://redis:6379/0. Não precisa de alteração no .env.
ALLOWED_ORIGINS	localhost:3000 → URL de produção	Flask aceita requisições CORS apenas de FRONTEND_URL. Em produção, como frontend e backend ficam no mesmo domínio (nginx roteia), o browser não envia preflight CORS, mas a configuração é necessária para casos de acesso direto ao backend.
WEB_CONCURRENCY	1 (dev) → 2+ (produção)	Múltiplos workers gunicorn. O APScheduler usa preload_app=True para rodar apenas no master e evitar e-mails duplicados.
Flask debug	DESLIGADO (padrão)	Em produção, app.debug=False. Erros 500 retornam mensagem genérica sem stack trace.

5.2 Frontend (.env.local / Build Args)

Item	Dev → Produção	Impacto
NEXT_PUBLIC_API_URL	http://127.0.0.1:5000 → https://seudominio.com/api	Em produção, o nginx roteia /api/ para Flask. A variável é embutida no JS no momento do build (não é possível mudar sem rebuildar).
NEXT_PUBLIC_SUPABASE_URL	Mesma URL do Supabase	Igual ao dev — o Supabase é gerenciado externamente.
output: standalone	Adicionado ao next.config.mjs	Gera um bundle Node.js mínimo sem node_modules completo, reduzindo a imagem Docker em ~70%.
CSP nonce (proxy.js)	REPORT_ONLY = false	CSP em modo bloqueio — scripts sem nonce são bloqueados pelo browser. Mitigação central de XSS e roubo de token.

6. Checklist Pós-Deploy

- [Todos os containers com status healthy: `docker compose ps`
- [`curl https://seudominio.com/api/health` retorna `{"status": "ok"}`
- [Login funciona via interface web (testado em navegador privado)
- [Criação de aluno com foto (webcam ou upload) funciona
- [Relatório PDF é gerado e baixado corretamente
- [Notificação de e-mail funciona (configure um aluno com vencimento próximo e aguarde)
- [Cloudflare: painel mostra tráfego passando (Analytics → Overview)
- [HTTPS funcionando: `https://` no URL, sem aviso de cert no browser
- [CSP ativa: F12 → Console deve estar limpo (sem 'Content Security Policy' errors)
- [`.env` NÃO está no repositório git: `git status | grep .env` não mostra o arquivo
- [Backup automático do banco configurado no painel Supabase (Point-in-Time Recovery)

7. Comandos de Manutenção

Atualizar a aplicação (novo código no repositório):

```
cd /opt/academia git pull origin main docker compose up -d --build
```

O build reutiliza as camadas em cache — só reconstroi o que mudou.

Ver logs em tempo real:

```
docker compose logs -f backend # logs do Flask docker compose logs -f frontend # logs do Next.js docker compose logs -f nginx # logs de acesso HTTP
```

Reiniciar um serviço específico:

```
docker compose restart backend docker compose restart frontend
```

Parar tudo (sem perder dados do Redis/volumes):

```
docker compose down
```

Limpar imagens antigas (liberar espaço após vários builds):

```
docker image prune -f docker system prune -f # cuidado: remove também containers parados
```

Ver uso de recursos (CPU, memória):

```
docker stats
```

Backup manual do Redis (dump.rdb):

```
docker compose exec redis redis-cli BGSAVE # O arquivo fica no volume redis_data  
# Para localizar: docker volume inspect gestao-academia_redis_data
```